

Report W7D1

Obiettivo

L'obiettivo di oggi e' di eseguire un set-up di un laboratorio per Python AI

Divideremo l'esercitazione in due parti, nella prima dove creiamo un ambiente separato (venv) ossia in Python è una cartella isolata che contiene una copia dell'interprete Python e dei pacchetti dedicati a uno specifico progetto, evitando conflitti con il sistema globale; Nella seconda parte dovremmo eseguire uno script in tal modo di comunicare con Gemini

Setup – Creazione dell'ambiente virtuale

Per prima cosa è stato creato un ambiente virtuale Python chiamato venv_ai utilizzando:

```
-python -m venv venv_ai
```

L'ambiente virtuale permette di mantenere separate le librerie utilizzate dal progetto rispetto a quelle installate globalmente nel sistema.

Successivamente l'ambiente è stato attivato tramite:

```
-source venv_ai/bin/activate
```

Una volta attivato, le librerie necessarie sono state installate utilizzando:

```
-pip install google-genai pandas numpy
```

In particolare, la libreria google-genai è quella necessaria per poter utilizzare le API di Gemini attraverso Python.

Per uscire dall'ambiente virtuale è possibile utilizzare:

```
Deactivate
```

Configurazione della Gemini API Key

Per poter comunicare con Gemini è stata utilizzata una API Key ottenuta tramite Google AI Studio.

La chiave API è stata inserita all'interno di una variabile d'ambiente chiamata:

GEMINI_API_KEY

Su Kali Linux la variabile è stata configurata tramite:

```
export GEMINI_API_KEY="incolla-qui-la-tua-api-key"
```

La chiave è stata inoltre aggiunta alla configurazione della shell tramite il file .zshrc, in modo da poter essere utilizzata anche nelle sessioni successive.

L'utilizzo di una variabile d'ambiente permette di evitare di inserire direttamente la API Key all'interno del codice Python.

Esercizio 1

Proviamo a interloquire con Gemini tramite domande fatte nello script

Iniziamo lo script importando dalla libreria google il modulo genai, mentre os è una libreria standard di Python che permette di interagire con il sistema operativo.

Nel nostro caso ci interessa soprattutto una cosa ossia leggere una variabile d'ambiente.

```
from google import genai
```

```
import os
```

Adesso tocca creare un client ossia l'oggetto che useremo per comunicare con Gemini

```
client = genai.Client(api_key=os.environ["GEMINI_API_KEY"])
```

Ora toccherà generare una risposta, in queste righe di comando, specifichiamo soprattutto il modello che ci risponderà, ossia Gemini-flash-latest e il contents e' quello che noi gli chiederemo a Gemini

```
response = client.models.generate_content(
```

```
model="gemini-flash-latest",
```

```
contents="Ciao! Chi sei?" )
```

e come comando finale per visualizzare la risposta useremo

```
print(response.text)
```